Funksionet

Programim në gjuhën C

Alion Emini

kurseinformatike.com

Përmbajtja

1. Hyrje
2. Përkufizimi dhe thirrja
   - Funksion që kthen vlerë
   - Funksion `void`
   - Funksion pa parametra
   - Forma e përgjithshme e përkufizimit
   - Thirrjet e funksioneve dhe vlera e kthyer
   - Shembull praktik: numri prim
3. Prototipet
4. Argumente dhe parametra
   - Kalimi sipas vlerës
   - Kur duam "dy rezultate": pointer-at
5. Array-t si argumente
   - Array 1D: pse duhet gjatësia
   - Modifikimi i array-t brenda funksionit
   - Array 2D dhe VLA (C99)

Përmbajtja

1. Hyrje
2. Përkufizimi dhe thirrja
3. Prototipet
4. Argumente dhe parametra
5. Array-t si argumente
6. Deklarata `return`
7. Mbyllje

Çfarë është një funksion?

Përkufizim

Një **funksion** është një seri deklaratash të grupuara dhe të emërtuara, që kryejnë një detyrë të caktuar.

Në C, një funksion:

- mund të ketë ose jo argumente;
- mund të kthejë ose jo një vlerë;
- është një "program i vogël" me deklarimet dhe deklaratat e veta.

Pse na duhen?

Funksionet e bëjnë kodin **modular**, **të ripërdorshëm** dhe më të **lehtë për mirëmbajtje**.

Avantazhet praktike të funksioneve

- **Ndarje e problemit**: programi ndahet në pjesë të vogla.
- **Ripërdorshmëri**: i njëjti funksion përdoret në shumë vende.
- **Shmang përsëritjen**: redukton "copy-paste".
- **Testim më i lehtë**: teston funksionin veçmas.
- **Lexueshmëri**: `is_prime(n)` është më e qartë se një bllok i gjatë në `main`.

Përmbajtja

1 Hyrje

2 Përkufizimi dhe thirrja
- Funksion që kthen vlerë
- Funksion `void`
- Funksion pa parametra
- Forma e përgjithshme e përkufizimit
- Thirrjet e funksioneve dhe vlera e kthyer
- Shembull praktik: numri prim

3 Prototipet

4 Argumente dhe parametra

Përmbajtja

1. Hyrje
2. Përkufizimi dhe thirrja
   - Funksion që kthen vlerë
   - Funksion `void`
   - Funksion pa parametra
   - Forma e përgjithshme e përkufizimit
   - Thirrjet e funksioneve dhe vlera e kthyer
   - Shembull praktik: numri prim
3. Prototipet
4. Argumente dhe parametra
   - Kalimi sipas vlerës
   - Kur duam "dy rezultate": pointer-at
5. Array-t si argumente
   - Array 1D: pse duhet gjatësia
   - Modifikimi i array-t brenda funksionit
   - Array 2D dhe VLA (C99)

Programi 1: funksion që llogarit mesataren

```
1 double mesatare(double a, double b) {
2     return (a + b) / 2;
3 }
```

Çfarë përfaqëson çdo pjesë?

- `double` në fillim: **tipi i kthimit**.
- a, b: **parametra** (si variabla lokale).
- `return ...`: kthen vlerën dhe përfundon funksionin.

Thirrja e funksionit mesatare

Thirrja është: emri + kllapa + argumente.

```
printf("Mesatarja: %g\n", mesatare(x, y));
avg = mesatare(x, y);

printf("%g\n", mesatare(5.1, 8.9));
printf("%g\n", mesatare(x / 2, y / 3));
```

Çfarë ndodh në thirrje?

Vlerat e argumenteve kopjohen te parametrat (`x -> a`, `y -> b`), pastaj ekzekutohet trupi dhe kthehet vlera.

Program i plotë: mesataret çift-pas-çifti

```c
/* Llogarit mesataret cift-pas-cifti te tre numrave */
#include <stdio.h>

double mesatare(double a, double b) {
    return (a + b) / 2;
}

int main(void) {
    double x, y, z;

    printf("Shkruani tre numra: ");
    scanf("%lf%lf%lf", &x, &y, &z);

    printf("Mesatarja e %g dhe %g: %g\n", x, y, mesatare(x, y));
    printf("Mesatarja e %g dhe %g: %g\n", y, z, mesatare(y, z));
    printf("Mesatarja e %g dhe %g: %g\n", x, z, mesatare(x, z));

    return 0;
}
```

Përmbajtja

1. Hyrje
2. Përkufizimi dhe thirrja
   - Funksion që kthen vlerë
   - Funksion `void`
   - Funksion pa parametra
   - Forma e përgjithshme e përkufizimit
   - Thirrjet e funksioneve dhe vlera e kthyer
   - Shembull praktik: numri prim
3. Prototipet
4. Argumente dhe parametra
   - Kalimi sipas vlerës
   - Kur duam "dy rezultate": pointer-at
5. Array-t si argumente
   - Array 1D: pse duhet gjatësia
   - Modifikimi i array-t brenda funksionit
   - Array 2D dhe VLA (C99)

Programi 2: funksion void (nuk kthen vlerë)

```c
void printo_numrimin(int n) {
    printf("T minus %d dhe duke numeruar\n", n);
}
```

Pikë kyçe

void do të thotë: funksioni **nuk kthen vlerë**. Thirrja e tij shfaqet si deklaratë më vete: `printo_numrimin(i);`.

Program i plotë: countdown me for

```c
/* Shtyp nje numerim zbrites */
#include <stdio.h>

void printo_numrimin(int n) {
    printf("T minus %d dhe duke numeruar\n", n);
}

int main(void) {
    int i;

    for (i = 10; i > 0; --i) {
        printo_numrimin(i);
    }

    return 0;
}
```

Përmbajtja

1. Hyrje
2. Përkufizimi dhe thirrja
   - Funksion që kthen vlerë
   - Funksion `void`
   - Funksion pa parametra
   - Forma e përgjithshme e përkufizimit
   - Thirrjet e funksioneve dhe vlera e kthyer
   - Shembull praktik: numri prim
3. Prototipet
4. Argumente dhe parametra
   - Kalimi sipas vlerës
   - Kur duam "dy rezultate": pointer-at
5. Array-t si argumente
   - Array 1D: pse duhet gjatësia
   - Modifikimi i array-t brenda funksionit
   - Array 2D dhe VLA (C99)

Programi 3: funksion pa parametra (void në listë)

```c
void printo_shaka(void) {
    printf("Te programosh ose jo te programosh: kjo eshte ceshtja.\n");
}
```

Pse shkruajmë void?

void brenda kllapave do të thotë: "ky funksion nuk pranon argumente". Kllapat **duhet** të jenë gjithmonë: printo_shaka();.

Testim i printo_shaka

```c
#include <stdio.h>

void printo_shaka(void) {
    printf("Te programosh ose jo te programosh: kjo eshte ceshtja.\n");
}

int main(void) {
    printo_shaka();
    return 0;
}
```

Përmbajtja

1. Hyrje
2. Përkufizimi dhe thirrja
   - Funksion që kthen vlerë
   - Funksion `void`
   - Funksion pa parametra
   - Forma e përgjithshme e përkufizimit
   - Thirrjet e funksioneve dhe vlera e kthyer
   - Shembull praktik: numri prim
3. Prototipet
4. Argumente dhe parametra
   - Kalimi sipas vlerës
   - Kur duam "dy rezultate": pointer-at
5. Array-t si argumente
   - Array 1D: pse duhet gjatësia
   - Modifikimi i array-t brenda funksionit
   - Array 2D dhe VLA (C99)

Struktura e një përkufizimi funksioni

Përkufizim

Forma e përgjithshme:

```
tipi_kthimit emri_funksionit(parametrat){ deklaratat }
```

Rregulla kryesore:

- tipi i kthimit **s'mund të mungojë** në C99;
- **array** nuk mund të jetë tip kthimi (por pointer mundet);
- `void` si tip kthimi ⇒ nuk kthen vlerë;
- `void` te parametrat ⇒ nuk pranon argumente.

Stil alternativ: tipi i kthimit në rresht më vete

```
1 double
2 mesatare(double a, double b) {
3     return (a + b) / 2;
4 }
```

Kur ndihmon?

Kur tipi i kthimit është i gjatë (p.sh. `unsigned long int`) ose për standard stilistik në projekt.

Deklarime brenda funksionit: C89 vs C99

- C89: deklarimet e variablave duhet të vijnë **në fillim** të trupit.
- C99: deklarimet mund të përzihen me deklarata ekzekutuese, por variabla duhet deklaruar **para përdorimit**.

```
double mesatare(double a, double b) {
    double shuma; /* deklarim */
    shuma = a + b; /* ekzekutim */
    return shuma / 2;
}
```

Funksion void me trup bosh (placeholder)

```c
void printo_shaka(void) {
    /* TODO: do e plotesojme me vone */
}
```

Pse përdoret?

Gjatë zhvillimit, mund të lëmë "skeletin" e funksionit dhe të plotësojmë trupin më vonë.

Përmbajtja

1. Hyrje
2. Përkufizimi dhe thirrja
   - Funksion që kthen vlerë
   - Funksion `void`
   - Funksion pa parametra
   - Forma e përgjithshme e përkufizimit
   - Thirrjet e funksioneve dhe vlera e kthyer
   - Shembull praktik: numri prim
3. Prototipet
4. Argumente dhe parametra
   - Kalimi sipas vlerës
   - Kur duam "dy rezultate": pointer-at
5. Array-t si argumente
   - Array 1D: pse duhet gjatësia
   - Modifikimi i array-t brenda funksionit
   - Array 2D dhe VLA (C99)

Thirrje me/pa vlerë kthimi

```c
/* funksion jo-void: mund te perdoret si shprehje */
avg = mesatare(x, y);
if (mesatare(x, y) > 0)
    printf("Mesatarja eshte pozitive\n");
printf("Mesatarja eshte %g\n", mesatare(x, y));

/* funksion void: thirrje si deklarate */
printo_numrimin(i);
printo_shaka();
```

Kur harrojmë kllapat: një gabim që duket "OK"

GABIM

```
printo_shaka; /* nuk thirret funksioni */
```

Pse është problem?

Pa kllapa, kjo është një shprehje "pa efekt". Disa kompilatorë japin paralajmërim.

Hedhja poshtë e vlerës së kthyer (opsionale)

Ndonjëherë kthen vlerë, por s'na intereson:

```
/* hedh poshte vleren e kthyer */
printf("Pershendetje, Mami!\n");

/* ose, për ta treguar qellimisht */
(void) printf("Pershendetje, Mami!\n");
```

Ideja

`printf` kthen numrin e karaktereve të printuara, por zakonisht nuk e ruajmë.

Përmbajtja

1. Hyrje
2. Përkufizimi dhe thirrja
   - Funksion që kthen vlerë
   - Funksion `void`
   - Funksion pa parametra
   - Forma e përgjithshme e përkufizimit
   - Thirrjet e funksioneve dhe vlera e kthyer
   - Shembull praktik: numri prim
3. Prototipet
4. Argumente dhe parametra
   - Kalimi sipas vlerës
   - Kur duam "dy rezultate": pointer-at
5. Array-t si argumente
   - Array 1D: pse duhet gjatësia
   - Modifikimi i array-t brenda funksionit
   - Array 2D dhe VLA (C99)

Pse ky shembull është i mirë?

- `main` bëhet i shkurtër dhe i qartë.
- Logjika e "prim-it" izolohet në `is_prime`.
- Funksioni kthen `bool` (true/false).

Programi 4: `is_prime` me `return` të shumëfishtë I

```c
/* Teston nese nje numer eshte prim */
#include <stdbool.h>
#include <stdio.h>

bool is_prime(int n) {
    int divisor;

    if (n <= 1)
        return false;

    for (divisor = 2; divisor * divisor <= n; divisor++)
        if (n % divisor == 0)
            return false;

    return true;
}
```

Programi 4: `is_prime` me `return` të shumëfishtë II

```c
int main(void) {
    int n;

    printf("Fut nje numer: ");
    scanf("%d", &n);

    if (is_prime(n))
        printf("Prim\n");
    else
        printf("Jo prim\n");

    return 0;
}
```

Shënime mbi `is_prime`

- Lejon **return** të shumtë: sapo gjendet pjestues, del menjëherë.
- Kontrollon vetëm deri te $\sqrt{n}$: nëse n ka pjestues $> \sqrt{n}$, ka edhe një $< \sqrt{n}$.
- Emrat e variablave mund të përsëriten në funksione të ndryshme (scope lokal).

Përmbajtja

1 Hyrje

2 Përkufizimi dhe thirrja

3 Prototipet

4 Argumente dhe parametra

5 Array-t si argumente

6 Deklarata `return`

7 Mbyllje

Problemi: thirrja para përkufizimit

Nëse thërrasim një funksion para se kompilatori të ketë parë prototipin/përkufizimin:

- kompilatori nuk e di tipin e kthimit, numrin e parametrave, tipet e tyre;
- në standardet e vjetra mund të ndodhin "supozime" të rrezikshme;
- në C99: thirrja pa deklaratë/përkufizim është **gabim**.

Rregull modern

Gjithmonë vendos prototipet përpara `main` (ose përpara thirrjes), zakonisht në header `.h`.

Prototipi i funksionit: forma

```
/* prototip */
double average(double a, double b);

/* ose pa emrat e parametrave */
double average(double, double);
```

Pse është i rëndësishëm?

Prototipi i jep kompilatorit informacion të plotë: kontrollon numrin/tipin e argumenteve dhe bën konvertimet e sakta.

Shembull: prototip + main + përkufizim

```c
#include <stdio.h>

/* DEKLARATA (PROTOTIPI) */
double average(double a, double b);

int main(void) {
    double x = 2.0, y = 4.0;
    printf("%g\n", average(x, y));
    return 0;
}

/* PERKUFIZIMI */
double average(double a, double b) {
    return (a + b) / 2;
}
```

Përmbajtja

1. Hyrje
2. Përkufizimi dhe thirrja
3. Prototipet
4. Argumente dhe parametra
   - Kalimi sipas vlerës
   - Kur duam "dy rezultate": pointer-at
5. Array-t si argumente
6. Deklarata `return`

Përmbajtja

1. Hyrje
2. Përkufizimi dhe thirrja
   - Funksion që kthen vlerë
   - Funksion `void`
   - Funksion pa parametra
   - Forma e përgjithshme e përkufizimit
   - Thirrjet e funksioneve dhe vlera e kthyer
   - Shembull praktik: numri prim
3. Prototipet
4. Argumente dhe parametra
   - Kalimi sipas vlerës
   - Kur duam "dy rezultate": pointer-at
5. Array-t si argumente
   - Array 1D: pse duhet gjatësia
   - Modifikimi i array-t brenda funksionit
   - Array 2D dhe VLA (C99)

Parametër vs argument

- **Parametrat** shfaqen te përkufizimi i funksionit (emra të përkohshëm).
- **Argumentet** shfaqen te thirrja (shprehje reale).

Rregulli i C-së

Argumentet kalohen **sipas vlerës**: funksioni merr **kopje** të vlerave. Ndryshimi i parametrave **nuk** ndryshon argumentet origjinale.

Shembull: power (kalim sipas vlerës)

```
int power(int x, int n) {
    int i, result = 1;
    for (i = 1; i <= n; i++)
        result = result * x;
    return result;
}
```

Optimizim: përdor parametrin si "counter" lokal

```
int power(int x, int n) {
    int result = 1;
    while (n-- > 0)
        result = result * x;
    return result;
}
```

Pse funksionon?

`n` brenda funksionit është kopje e argumentit. E modifikojmë pa dëmtuar vlerën origjinale në `main`.

Përmbajtja

1. Hyrje
2. Përkufizimi dhe thirrja
   - Funksion që kthen vlerë
   - Funksion `void`
   - Funksion pa parametra
   - Forma e përgjithshme e përkufizimit
   - Thirrjet e funksioneve dhe vlera e kthyer
   - Shembull praktik: numri prim
3. Prototipet
4. Argumente dhe parametra
   - Kalimi sipas vlerës
   - Kur duam "dy rezultate": pointer-at
5. Array-t si argumente
   - Array 1D: pse duhet gjatësia
   - Modifikimi i array-t brenda funksionit
   - Array 2D dhe VLA (C99)

Kur kalimi sipas vlerës nuk mjafton

Ndonjëherë duam që funksioni të modifikojë variabla jashtë tij (p.sh. të kthejë dy vlera). Zgjidhja në C: kalojmë **adresat** (pointer-a).

Ideja

Në vend që të kthejmë dy vlera me `return`, kalojmë dy pointer-a dhe modifikojmë `*pointer`.

Shembull: decompose me pointer-a

```c
void decompose(double x, long *int_part, double *frac_part) {
    *int_part = (long) x;
    *frac_part = x - *int_part;
}

/* thirrja */
decompose(3.14159, &i, &d);
```

Kur argumentet nuk përputhen me parametrat

Rregullat varen nga prania e prototipit:

- **Me prototip**: argumentet konvertohen si në caktim (p.sh. `int -> double`).
- **Pa prototip**: bëhen "promovimet e paracaktuara" (p.sh. `float -> double`, `char/short -> int`).

Pse është e rrezikshme?

Pa prototip, thirrja mund të shkaktojë sjellje të papërcaktuar nëse funksioni pret tip tjetër nga ai që po i jepet.

Shembull problematik (pa prototip)

```c
#include <stdio.h>

int main(void) {
    double x = 3.0;
    printf("Square: %d\n", square(x)); /* rrezik */
    return 0;
}

int square(int n) {
    return n * n;
}
```

Zgjidhja

Vendos prototipin **para** `main` (ose bëj cast, por prototipi është zgjidhja e duhur).

Përmbajtja

1. Hyrje
2. Përkufizimi dhe thirrja
3. Prototipet
4. Argumente dhe parametra
5. Array-t si argumente
   - Array 1D: pse duhet gjatësia
   - Modifikimi i array-t brenda funksionit
   - Array 2D dhe VLA (C99)
   - Literale të përbëra (C99)

Përmbajtja

1. Hyrje
2. Përkufizimi dhe thirrja
   - Funksion që kthen vlerë
   - Funksion `void`
   - Funksion pa parametra
   - Forma e përgjithshme e përkufizimit
   - Thirrjet e funksioneve dhe vlera e kthyer
   - Shembull praktik: numri prim
3. Prototipet
4. Argumente dhe parametra
   - Kalimi sipas vlerës
   - Kur duam "dy rezultate": pointer-at
5. Array-t si argumente
   - Array 1D: pse duhet gjatësia
   - Modifikimi i array-t brenda funksionit
   - Array 2D dhe VLA (C99)

Array si parametër: gjatësia nuk "vjen" bashkë

- Parametri `int a[]` pranon një array 1D.
- Por funksioni **s'ka mënyrë** të dijë sa elementë ka argumenti.
- Prandaj zakonisht kalojmë edhe **gjatësinë** `n`.

Shembull: sum_array

```c
int sum_array(int a[], int n) {
    int i, sum = 0;
    for (i = 0; i < n; i++)
        sum += a[i];
    return sum;
}

/* prototip */
int sum_array(int a[], int n);
```

Thirrja dhe gabimi tipik

```c
#define LEN 100

int main(void) {
    int b[LEN], total;
    /* ... */
    total = sum_array(b, LEN); /* OK */
    /* ... */
}

/*** GABIM ***/
total = sum_array(b[], LEN);
```

Kujdes me gjatësinë që i jep funksionit

Rrezik

Nëse i jep `n` më të madh se gjatësia reale e të dhënave, funksioni kalon fundin e array-t $\Rightarrow$ sjellje e papërcaktuar.

Përdorim korrekt

Nëse janë vetëm 50 elementë të vlefshëm: `sum_array(b, 50);` është i pranueshëm.

Përmbajtja

1. Hyrje
2. Përkufizimi dhe thirrja
   - Funksion që kthen vlerë
   - Funksion `void`
   - Funksion pa parametra
   - Forma e përgjithshme e përkufizimit
   - Thirrjet e funksioneve dhe vlera e kthyer
   - Shembull praktik: numri prim
3. Prototipet
4. Argumente dhe parametra
   - Kalimi sipas vlerës
   - Kur duam "dy rezultate": pointer-at
5. Array-t si argumente
   - Array 1D: pse duhet gjatësia
   - Modifikimi i array-t brenda funksionit
   - Array 2D dhe VLA (C99)

Funksion që modifikon array-n

```
1 void store_zeros(int a[], int n) {
2     int i;
3     for (i = 0; i < n; i++)
4         a[i] = 0;
5 }
6
7 /* thirrja */
8 store_zeros(b, 100);
```

Pse reflektohet jashtë?

Emri i array-t në thirrje "shndërrohet" në adresën e elementit të parë; funksioni punon mbi të njëjtat qeliza memorieje.

Përmbajtja

1. Hyrje
2. Përkufizimi dhe thirrja
   - Funksion që kthen vlerë
   - Funksion `void`
   - Funksion pa parametra
   - Forma e përgjithshme e përkufizimit
   - Thirrjet e funksioneve dhe vlera e kthyer
   - Shembull praktik: numri prim
3. Prototipet
4. Argumente dhe parametra
   - Kalimi sipas vlerës
   - Kur duam "dy rezultate": pointer-at
5. Array-t si argumente
   - Array 1D: pse duhet gjatësia
   - Modifikimi i array-t brenda funksionit
   - Array 2D dhe VLA (C99)

Array multidimensional: dimensionet pasues duhet të dihen

- Te 2D, vetëm dimensioni i parë mund të mungojë në parametër.
- Numri i kolonave duhet të jetë i njohur (ose të jepet si parametër me VLA).

Shembull 2D: duhet numri i kolonave

```c
#define LEN 10

int sum_two_dimensional_array(int a[][LEN], int n) {
    int i, j, sum = 0;
    for (i = 0; i < n; i++)
        for (j = 0; j < LEN; j++)
            sum += a[i][j];
    return sum;
}
```

VLA (C99): gjatësia varet nga parametra

```c
int sum_array(int n, int a[n]) {
    int i, sum = 0;
    for (i = 0; i < n; i++)
        sum += a[i];
    return sum;
}
```

Detaj

Rendi i parametrave ka rëndësi: `n` duhet të vijë para `a[n]`.

Përmbajtja

1. Hyrje
2. Përkufizimi dhe thirrja
   - Funksion që kthen vlerë
   - Funksion `void`
   - Funksion pa parametra
   - Forma e përgjithshme e përkufizimit
   - Thirrjet e funksioneve dhe vlera e kthyer
   - Shembull praktik: numri prim
3. Prototipet
4. Argumente dhe parametra
   - Kalimi sipas vlerës
   - Kur duam "dy rezultate": pointer-at
5. Array-t si argumente
   - Array 1D: pse duhet gjatësia
   - Modifikimi i array-t brenda funksionit
   - Array 2D dhe VLA (C99)

Literal i përbërë: array "në fluturim"

```
total = sum_array((int[]){3, 0, 3, 4, 1}, 5);
total = sum_array((int[]){2 * i, i + j, j * k}, 3);
```

Kur është i dobishëm?

Kur do të testosh shpejt një funksion pa krijuar një array me emër të veçantë.

Përmbajtja

1 Hyrje

2 Përkufizimi dhe thirrja

3 Prototipet

4 Argumente dhe parametra

5 Array-t si argumente

6 Deklarata `return`

7 Mbyllje

`return`: kthimi i vlerës dhe përfundimi i funksionit

- Funksion jo-`void`: `return shprehje;`
- Funksion `void`: lejohet `return;` (pa shprehje).
- Shprehja konvertohet automatikisht në tipin e kthimit, nëse duhet.

Shembuj

- `return 0;`
- `return status;`
- `return n >= 0 ? n : 0;`

`return` në funksion `void` për dalje të parakohshme

```c
void printo_int(int i) {
    if (i < 0)
        return; /* del pa printuar */
    printf("%d", i);
}
```

Kujdes

Në funksion `void`, `return shprehje;` është gabim kompilimi.

Kur një funksion jo-`void` "bie" nga fundi

Sjellje e papërcaktuar

Nëse një funksion jo-`void` arrin fundin pa `return`, dhe programi përpiqet të përdorë "vlerën e kthyer", rezultati është i papërcaktuar.

Praktikë e mirë

Sigurohu që çdo rrugë ekzekutimi në funksion të përfundojë me `return` kur tipi i kthimit nuk është `void`.

`main` dhe kodi i statusit

- `main` është funksion: zakonisht `int main(void)`.
- `return 0;` ⇒ përfundim normal.
- `return != 0;` ⇒ përfundim jonormal (varësisht OS).

Stil modern

Në C99 është gabim të mungojë tipi i kthimit. Shkruaj qartë `int main(void)`.

Funksioni exit (nga <stdlib.h>)

```
#include <stdlib.h>

exit(0); /* perfundim normal */
exit(EXIT_SUCCESS); /* perfundim normal */
exit(EXIT_FAILURE); /* perfundim jonormal */
```

Çfarë ndryshon nga return?

`return` përfundon programin vetëm kur është në `main`. `exit` e përfundon programin nga **çdo funksion**.

Përmbajtja

1 Hyrje

2 Përkufizimi dhe thirrja

3 Prototipet

4 Argumente dhe parametra

5 Array-t si argumente

6 Deklarata `return`

7 Mbyllje

Çfarë mësove në këtë leksion

- Përkufizimi i funksioneve: tipi i kthimit, emri, parametrat, trupi
- Thirrja e funksioneve: argumente, kllapa, funksione `void`
- Shembuj praktikë: `mesatare`, `printo_numrimin`, `printo_shaka`, `is_prime`
- Prototipet: pse duhen dhe si shmangin gabime
- Argumente vs parametra, kalimi sipas vlerës, pointer-a për rezultate të shumta
- Array-t si argumente (1D/2D), VLA dhe literalet e përbëra (C99)
- `return`: rregullat për `void` dhe jo-`void`
- Përfundimi i programit: `return` në `main` vs `exit`

Çfarë vjen më pas (sugjerim)

- **Scope** dhe **storage class** (lokale, globale, `static`)
- **Header files** (`.h`) dhe organizimi i projekteve me shumë skedarë
- **Pointers** më në thellësi (pass-by-reference pattern, string, array)

Faleminderit për vëmendjen!

Pyetje?